Sysinternals Quick Reference

What to run, what in the output matters, and the native fallback, ordered by the five questions

Use Sysinternals when it's on the bench. It sees more than the built-ins (signatures, hashes, every autostart location at once). Every tool has a native fallback in the Execution Reference, so nothing here is a single point of failure.

Footprint. First run of each tool writes an EULA-accepted key under `HKCU\Software\Sysinternals`. Pass `-accepteula` and note it in Hunt Notes ("ran autorunsc, EULA key written"). Run from your tools media, write output to your output path, never to the evidence volume. Everything below is read-only. **Never** run `sdelete`, `psexec` against other hosts, or `pskill` on evidence.

Run order on a live Windows host (after memory capture):

```
1 autorunsc  (Q3)   2 sigcheck (Q1)   3 tcpvcon (Q2)   4 pslist -t / Process Explorer (Q1)
5 listdlls -u / handle on suspects (Q1)   6 logonsessions / psloggedon (Q4)   7 streams / strings on
files (Q1/Q5)
```

Q1 — What is running that should not be?

Process Explorer (`procexp64.exe` , GUI)

- `Options → Verify Image Signatures`. Add columns (`View → Select Columns`): **Image Path, Command Line, Verified Signer, Company Name, Start Time**.
- Tree view = parentage. Look for Office/browser → `cmd` / `powershell`, `svchost` not under `services.exe`, `lsass` not under `wininit`.
- Colors: **purple** = packed/compressed image, **pink** = service, **light blue** = same user as you.
- Suspect PID → lower pane (`Ctrl+L`): **DLL view** (unsigned or user-path DLLs) and **Handle view** (files, mutexes, keys).
- Properties → **TCP/IP** tab (its connections), **Strings** tab (Image vs Memory; memory strings reveal unpacked content).
- Fallback: Execution Reference §1.2.

PsList (CLI)

```
pslist -accepteula -t           # tree with parent/child
pslist -accepteula -x <PID>     # threads + memory detail for one PID
```

Sigcheck: unsigned executables in user-writable paths, with hashes:

```
sigcheck64 -accepteula -nobanner -e -u -s -h -c C:\Users C:\ProgramData C:\Windows\Temp > sigcheck.csv
```

`-e` executables only · `-u` unsigned only · `-s` recurse · `-h` hashes · `-c` CSV. Look at Path + Publisher + SHA256. Anything unsigned in `AppData`, `Public`, `ProgramData`, `Temp` is a Q1 lead. Fallback: §1.2 `Get-AuthenticodeSignature`.

ListDLLs: unsigned DLLs loaded anywhere (DLL hijack / injection):

```
listdlls64 -accepteula -u               # unsigned DLLs, all processes
listdlls64 -accepteula <PID|name>       # everything one process loaded
```

Fallback: Vol3 `windows.dlllist --pid` / `windows.ldrmodules`.

Handle: what a suspect has open:

```
handle64 -accepteula -nobanner -p <PID>          # all handles for the PID
handle64 -accepteula -nobanner -a <name-fragment>  # who holds a file/mutex/key matching a string
```

Fallback: Vol3 `windows.handles --pid`.

Q2 — What is talking that should not be?

TCPView (GUI) / **tcpvcon** (CLI): connection → process, with the owning image path:

```
tcpvcon64 -accepteula -a -c -n > tcpvcon.csv     # all endpoints, CSV, no name resolution
```

Look for: established sessions owned by processes with no network role (`notepad`, `rundll32`, Office), listening ports above 1024 owned by user-path binaries, the same remote IP from two hosts. Fallback: §1.3 `Get-NetTCPConnection` / `netstat -anob`; memory: `windows.netscan`.

Q3 — How does it survive a reboot?

Autoruns (GUI) / **autorunsc** (CLI): every autostart location in one pass. This is the highest-value Sysinternals tool for a hunt.

```
autorunsc64 -accepteula -nobanner -a * -c -h -s -m -t > autoruns.csv
```

`-a *` all categories · `-c` CSV · `-h` hashes · `-s` verify signatures · `-m` hide Microsoft-signed · `-t` timestamps in **UTC** (straight into Hunt Notes).

Read the CSV for: **Signer** = `(Not verified)`, **Image Path** in user-writable dirs, **Launch String** with `powershell -enc`, `rundll32 <odd>.dll`, `mshta`, `wscript`, `cmd /c` + a script in ProgramData.

Categories that matter: Logon (Run keys), Services, Scheduled Tasks, WMI, Winlogon, Image Hijacks (IFEO), AppInit, Known DLLs, Boot Execute.

GUI: `Options → Hide Microsoft Entries + Verify Code Signatures`. **Pink** = no publisher / unsigned, **yellow** = entry exists but file is missing (deleted tooling: still a Q3/Q5 lead). `Entry → Jump to Entry` opens the key or task.

Fallback: §1.4 (Run keys, services, ServiceDll, tasks, WMI, Winlogon, IFEO, BITS).

PsService: one service in detail:

```
psservice64 -accepteula config <ServiceName>
```

AccessChk: weak permissions that let a low-priv user plant persistence or escalate:

```
accesschk64 -accepteula -nobanner -uwcqv "Authenticated Users" *     # services anyone can reconfigure
accesschk64 -accepteula -nobanner -uwsq Users "C:\Program Files"      # writable program dirs
```

A writable service binary path or service config is a Q3/Q4 finding (and a hardening recommendation).

Q4 — Who did it, and how did they get in?

```
logonsessions64 -accepteula -nobanner -p     # every logon session + its processes (logon type, auth package)
psloggedon64 -accepteula                     # local and resource-share logons
psinfo64 -accepteula -s -h                   # OS build, install date, software, hotfixes
```

`logonsessions -p` ties a process to the session that started it: a network (type 3) or RemoteInteractive (type 10) session owning the Q1 process means the operator reached in. Fallback: §1.5 + §1.7 (4624/4648/4672).

Q5 — When did it start? / hidden content

Streams: Alternate Data Streams (hidden payloads, `Zone.Identifier` download origin):

```
streams64 -accepteula -nobanner -s C:\Users\Public
streams64 -accepteula -nobanner <file>
```

`Zone.Identifier` with `ZoneId=3 + HostUrl=` tells you where a file was downloaded from. Read it natively: `Get-Content <file> -Stream Zone.Identifier`.

Strings: quick static look at a suspect binary (export it first; don't run it):

```
strings64 -accepteula -nobanner -n 8 <file> | findstr /i "http :// .onion cmd powershell \\pipe\\
HKEY"
```

Sigcheck for file metadata + hashes: `sigcheck64 -accepteula -nobanner -a -h <file>` (`-a` adds original/internal name and entropy; high entropy = packed).

Process Monitor (`procmon64.exe`): live behavior, **only** for a short, deliberate capture:

- Filter first (`Ctrl+L`): `Process Name is <suspect>` → Include. Or `Operation is Process Create` , `RegSetValue` , `WriteFile` .
- `Tools → Process Tree` (lineage + start/end times for the capture), `Tools → File Summary` / `Registry Summary` .
- Footprint: loads a driver and generates heavy I/O. Capture a minute, save `.PML` to the output path, stop. Note start/stop UTC in Hunt Notes.

Sysmon: installing it changes the host. Do it only if authorized and noted. Reading an existing Sysmon log is in §1.7.

Output → Hunt Notes

Tool output	Hunt Notes entry
autorunsc row (unsigned, odd path)	Q3, source `autorunsc` , time = its UTC timestamp, IOC = image path + SHA-256
sigcheck row	Q1, source `sigcheck` , IOC = path + hash
tcpvcon row	Q2, source `tcpvcon` , IOC = remote IP:port, evidence = owning PID/image
logonsessions	Q4, source `logonsessions` , IOC = account, evidence = logon type + session ID
Zone.Identifier	Q4/Q5, source `streams` , IOC = HostUrl

A Sysinternals result plus a memory (Volatility) or disk (FTK/Autopsy) result for the same item = **two independent sources**.